

# FutureKawa

## Documentation Technique

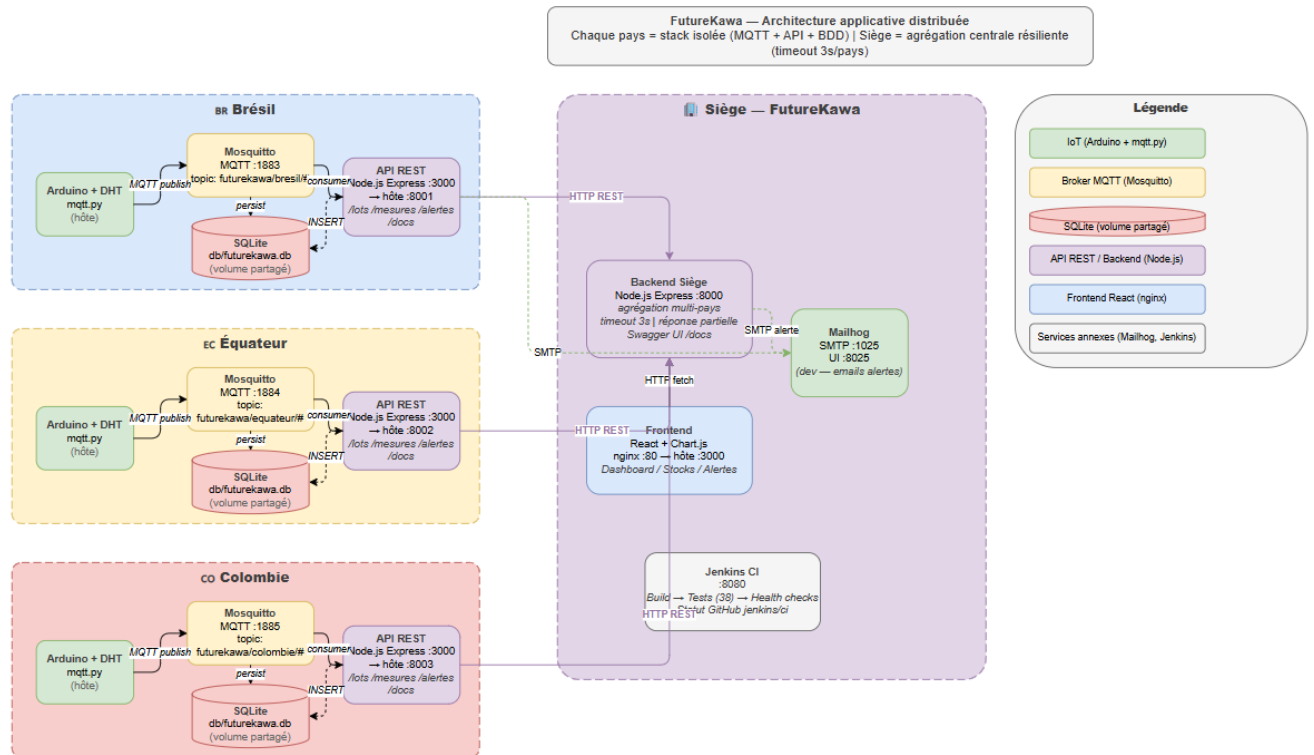
*MSPR Bloc 4 — RNCP35584 — EPSI 2025-2026*

| Projet                                         | Équipe      | Version | Date      |
|------------------------------------------------|-------------|---------|-----------|
| FutureKawa — Suivi IoT des stocks de café vert | Bloc 4 EPSI | 1.0     | Juin 2026 |

# 1. Architecture globale

## 1.1 Vue d'ensemble

FutureKawa est une architecture distribuée multi-pays : chaque pays producteur (Brésil, Équateur, Colombie) dispose d'une stack indépendante et conteneurisée. Le siège agrège les données de tous les pays via un backend central.



## 1.2 Composants par pays

| Composant       | Technologie             | Rôle                                  |
|-----------------|-------------------------|---------------------------------------|
| Broker MQTT     | Eclipse Mosquitto 2.0   | Réception des messages IoT            |
| API REST        | Node.js 20 + Express 5  | Gestion lots, mesures, alertes        |
| Base de données | SQLite (better-sqlite3) | Persistance partagée entre les 3 APIs |
| IoT bridge      | Python 3 + paho-mqtt    | Lecture Arduino → publication MQTT    |
| Capteur         | Arduino UNO + DHT11/22  | Mesure température et humidité        |

## 1.3 Composants siège

| Composant     | Technologie            | Rôle                                     |
|---------------|------------------------|------------------------------------------|
| Backend siège | Node.js 20 + Express 5 | Agrégation et proxy vers les APIs pays   |
| Frontend      | React 18 + Chart.js 4  | Interface de consultation                |
| Serveur web   | nginx 1.27             | Serveur de production + proxy            |
| Email dev     | Mailhog                | Interception des emails en développement |

## 1.4 Infrastructure Docker

La stack est entièrement conteneurisée via Docker Compose.

### Fichiers de configuration :

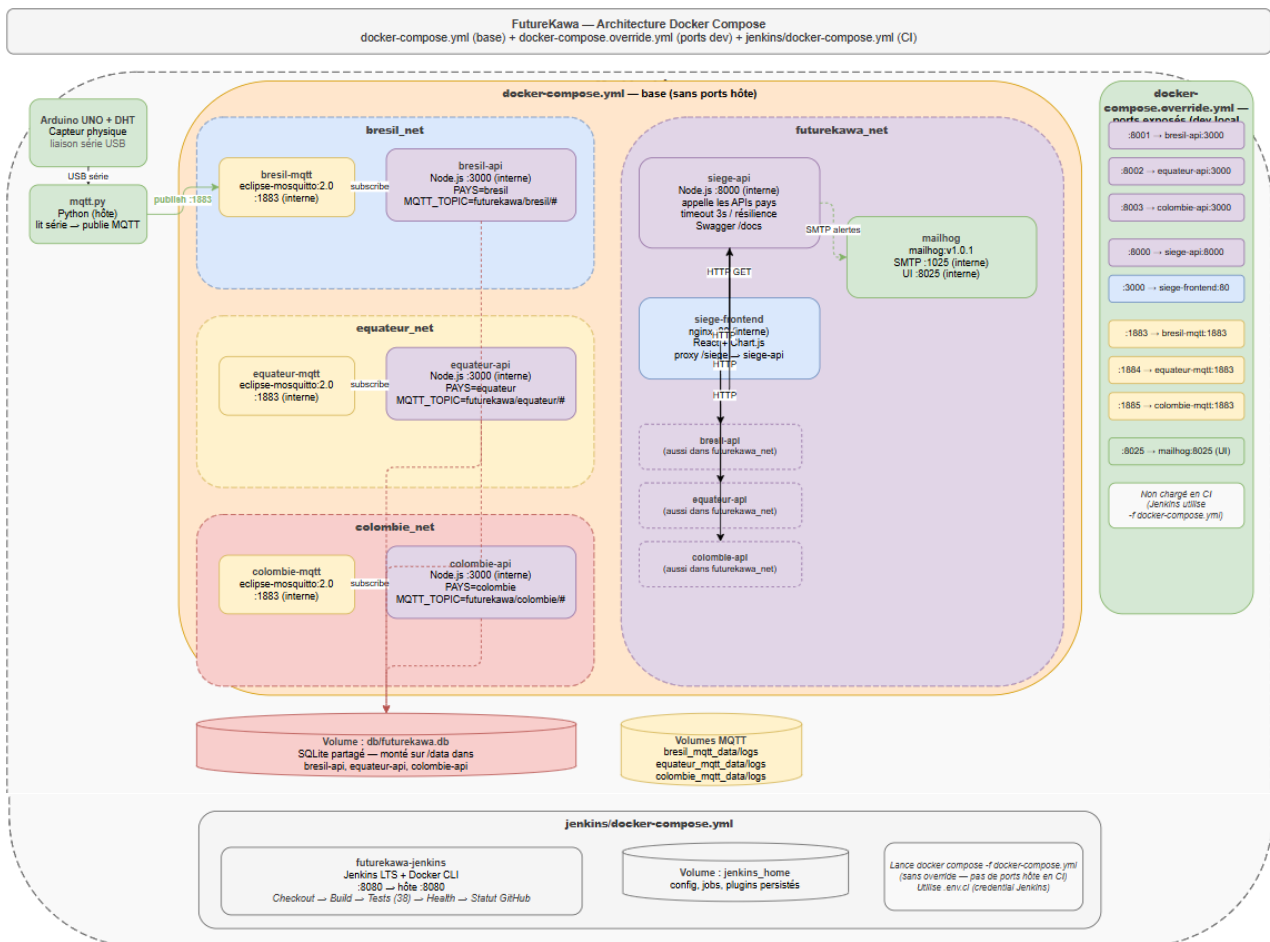
| Fichier                     | Rôle                                  |
|-----------------------------|---------------------------------------|
| docker-compose.yml          | Base : services, réseaux, volumes     |
| docker-compose.override.yml | Dev local : ports exposés vers l'hôte |
| jenkins/docker-compose.yml  | Jenkins CI isolé                      |

### Réseaux Docker :

| Réseau         | Services connectés                                       |
|----------------|----------------------------------------------------------|
| bresil_net     | bresil-mqtt, bresil-api                                  |
| equateur_net   | equateur-mqtt, equateur-api                              |
| colombie_net   | colombie-mqtt, colombie-api                              |
| futurekawa_net | toutes les APIs pays, siege-api, siege-frontend, mailhog |

### Ports exposés (développement local) :

| Service        | Port hôte | Port interne |
|----------------|-----------|--------------|
| bresil-api     | 8001      | 3000         |
| equateur-api   | 8002      | 3000         |
| colombie-api   | 8003      | 3000         |
| siege-api      | 8000      | 8000         |
| siege-frontend | 3000      | 80           |
| bresil-mqtt    | 1883      | 1883         |
| equateur-mqtt  | 1884      | 1883         |
| colombie-mqtt  | 1885      | 1883         |
| mailhog UI     | 8025      | 8025         |
| Jenkins        | 8080      | 8080         |



## 1.5 Flux de données

### Flux IoT → BDD :

1. Arduino mesure température et humidité toutes les 30 secondes
2. Le script mqtt.py lit la mesure via liaison série USB
3. Il publie un message JSON sur le topic MQTT du pays
4. Le consumer MQTT de l'API pays reçoit le message
5. Il compare les valeurs aux seuils de l'entrepôt
6. Il insère la mesure en BDD et crée une alerte si hors plage

### Flux consultation siège :

7. Le frontend appelle le backend siège (ex. GET /siege/stocks)
8. Le siège appelle en parallèle les 3 APIs pays
9. Chaque pays répond avec ses données ou une erreur timeout
10. Le siège consolide et retourne une réponse partielle si nécessaire
11. Le frontend affiche les données avec les graphiques Chart.js

## 1.6 Résilience et tolérance aux pannes

### Principe de résilience du siège :

Le backend siège est conçu pour ne jamais bloquer si un pays est indisponible. Chaque appel vers une API pays est soumis à un timeout configurable (3 secondes par défaut). En cas de panne ou de timeout, la réponse inclut une indication d'erreur partielle sans bloquer les autres pays.

```

{
  "total_pays": 3,
  "pays_ok": 2,
  "pays_en_erreur": 1,
  "resultats": [
    { "pays": "bresil", "status": "ok", "data": [...] },
    { "pays": "equateur", "status": "ok", "data": [...] },
    { "pays": "colombie", "status": "error", "error": "Timeout (>3s)" }
  ]
}

```

### Résilience du consumer MQTT :

Le consumer MQTT gère automatiquement les déconnexions via la bibliothèque mqtt.js :

- Reconnexion automatique sur coupure réseau
- Log des erreurs de connexion (console.error)
- Reprise de la souscription aux topics après reconnexion

### Résilience de l'IoT :

Le script mqtt.py ignore les lignes de lecture série invalides (format inattendu, erreur DHT) et continue la boucle sans planter.

## 2. Justification des choix techniques

### 2.1 Pourquoi une architecture microservices ?

Le cahier des charges impose une architecture distribuée pays ↔ siège. Ce découpage répond à plusieurs contraintes métier :

| Besoin métier                                    | Réponse architecturale               |
|--------------------------------------------------|--------------------------------------|
| Chaque pays doit fonctionner de manière autonome | Stack indépendante par pays          |
| Le siège doit consolider sans dépendre d'un pays | Agrégation avec tolérance aux pannes |
| Les seuils varient par pays                      | Configuration en BDD par entrepôt    |
| Déploiement progressif (un pays à la fois)       | Docker Compose par pays              |

Une architecture monolithique aurait rendu impossible le fonctionnement autonome d'un pays en cas de panne du siège.

### 2.2 Pourquoi MQTT plutôt qu'HTTP pour l'IoT ?

| Critère                      | MQTT                        | HTTP                          |
|------------------------------|-----------------------------|-------------------------------|
| Consommation mémoire Arduino | Très faible                 | Élevée                        |
| Fiabilité réseau instable    | Publication asynchrone, QoS | Connexion synchrone bloquante |
| Découplage capteur / serveur | Total (publish/subscribe)   | Couplage fort                 |
| Reconnexion automatique      | Native                      | À implémenter manuellement    |

MQTT est le standard industriel pour l'IoT. La bibliothèque PubSubClient (Arduino) et paho-mqtt (Python) gèrent nativement la reconnexion et le mode hors-ligne.

## 2.3 Pourquoi Mosquitto ?

- Image Docker officielle légère (eclipse-mosquitto:2.0)
- Configuration simple par fichier texte
- Standard de fait pour les brokers MQTT embarqués
- Compatible avec tous les clients MQTT (Arduino, Python, Node.js)

## 2.4 Pourquoi Node.js + Express ?

- Écosystème npm riche pour MQTT, email, SQLite
- Modèle async natif adapté aux I/O multiples (MQTT + HTTP + BDD)
- 
- FastAPI (initialement envisagé) apportait une dépendance Python supplémentaire redondante avec le script IoT
- Express 5 avec gestion d'erreurs async native

## 2.5 Pourquoi SQLite plutôt que PostgreSQL ?

Choix pragmatique justifié par le contexte MSPR (développement rapide du projet) :

| Critère                    | SQLite                     | PostgreSQL                     |
|----------------------------|----------------------------|--------------------------------|
| Configuration              | Zéro (fichier)             | Service + auth + init          |
| Volume partagé Docker      | Fichier monté              | Connexion réseau + credentials |
| Performance (< 1000 req/s) | Suffisant                  | Sur-dimensionné                |
| Migrations                 | CREATE TABLE IF NOT EXISTS | Outils dédiés                  |

Pour une mise en production réelle, PostgreSQL serait préféré pour la concurrence et les performances à grande échelle.

## 2.6 Pourquoi React + Chart.js ?

- React : composants réutilisables, hooks pour les appels API
  - Chart.js 4 : courbes temporelles avec annotations de seuils
  - Vite : build ultra-rapide, hot reload en développement
  - nginx : serveur de production léger avec reverse proxy intégré
-

## 3. Conception IoT

### 3.1 Matériel utilisé

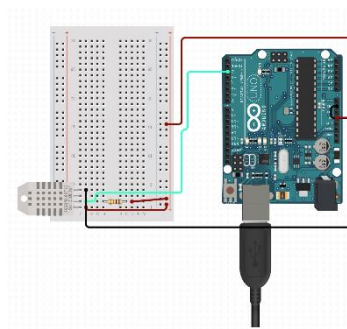
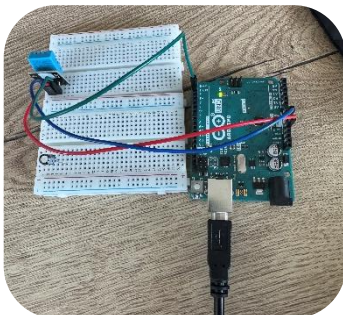
| Composant       | Référence        | Rôle                                 |
|-----------------|------------------|--------------------------------------|
| Microcontrôleur | Arduino UNO      | Lecture capteur + transmission série |
| Capteur         | DHT11 / DHT22    | Mesure température et humidité       |
| Résistance      | 10 kΩ (pull-up)  | Stabilisation signal DATA du DHT     |
| Câble USB       | USB-A vers USB-B | Liaison série Arduino ↔ PC           |
| PC hôte         | Tout OS          | Exécution de mqtt.py                 |

#### Précision des capteurs :

| Capteur | Précision température     | Précision humidité |
|---------|---------------------------|--------------------|
| DHT11   | $\pm 2^{\circ}\text{C}$   | $\pm 5\%$          |
| DHT22   | $\pm 0.5^{\circ}\text{C}$ | $\pm 2\%$          |

*[i] Le DHT22 est recommandé pour les seuils du projet ( $\pm 3^{\circ}\text{C}$  /  $\pm 2\%$ ) car le DHT11 peut être insuffisant en précision humidité.*

### 3.2 Schéma de câblage



#### Connexions :

| Pin DHT | Connexion                                |
|---------|------------------------------------------|
| VCC     | 5V Arduino                               |
| DATA    | Pin digital 2 + résistance 10kΩ vers VCC |
| GND     | GND Arduino                              |

### 3.3 Code Arduino

Le programme Arduino lit le capteur DHT toutes les 2 secondes et envoie les valeurs sur le port série au format texte :

```
54.20 28.50
```

*(humidité puis température, séparés par un espace)*

```

#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float h = dht.readHumidity();
  float t = dht.readTemperature();

  if (isnan(h) || isnan(t)) {
    Serial.println("Erreur lecture capteur");
    return;
  }

  Serial.print(h);
  Serial.print(" ");
  Serial.println(t);

  delay(2000);
}

```

### 3.4 Structure des topics MQTT

Format : futurekawa/{pays}/{entrepot\_code}/mesures

| Exemple de topic                 | Description                |
|----------------------------------|----------------------------|
| futurekawa/bresil/BR01/mesures   | Entrepôt São Paulo, Brésil |
| futurekawa/equateur/EQ01/mesures | Entrepôt Quito, Équateur   |
| futurekawa/colombie/CO01/mesures | Entrepôt Bogotá, Colombie  |

Les APIs pays souscrivent au topic générique futurekawa/{pays}/# pour recevoir tous les entrepôts du pays.

### 3.5 Format du payload JSON

```

{
  "entrepot": "BR01",
  "temp": 28.5,
  "hum": 54.2,
  "ts": "2026-05-28T14:13:17.031326+00:00"
}

```

| Champ    | Type   | Description                        |
|----------|--------|------------------------------------|
| entrepot | string | Code de l'entrepôt (BR01, EQ02...) |
| temp     | float  | Température en °C                  |
| hum      | float  | Humidité relative en %             |



| Champ | Type            | Description                 |
|-------|-----------------|-----------------------------|
| ts    | string ISO 8601 | Horodatage UTC de la mesure |

### 3.6 Fréquence d'envoi

| Paramètre                  | Valeur                              | Justification               |
|----------------------------|-------------------------------------|-----------------------------|
| Fréquence Arduino          | 1 lecture / 2s                      | Limite recommandée DHT      |
| Fréquence publication MQTT | À chaque lecture valide             | Données fraîches en continu |
| Tolérance dérive           | $\pm 3^{\circ}\text{C}$ / $\pm 2\%$ | Définie par pays en BDD     |

### 3.7 Stratégie de reconnexion

#### Reconnexion MQTT (script Python) :

La bibliothèque paho-mqtt gère automatiquement la reconnexion via `loop_start()`. En cas de coupure réseau, le client tente de se reconnecter toutes les quelques secondes jusqu'au rétablissement.

#### Reconnexion MQTT (consumer Node.js) :

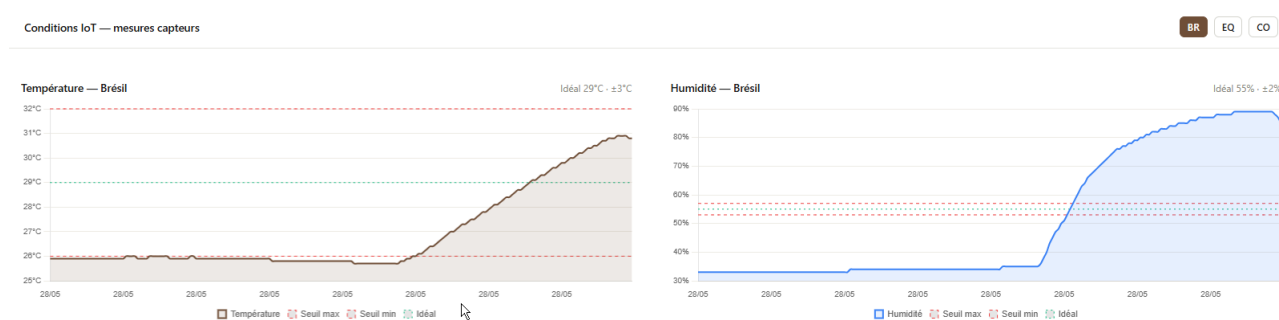
L'événement `offline` est loggué. La bibliothèque `mqtt.js` tente automatiquement la reconnexion avec `backoff exponentiel`.

#### Gestion des erreurs de lecture DHT :

Le script `mqtt.py` ignore les lignes contenant "Erreur" ou dont le format est invalide :

```
if len(parts) < 2 or "Erreur" in parts[0]:
    continue
```

### 3.8 Flux end-to-end validé



```
Lecture série sur /dev/cu.usbmodem11201...
MQTT connecté (Success) - topic : futurekawa/equateur/EQ01/mesures
Publié → futurekawa/equateur/EQ01/mesures : {"entrepot": "EQ01", "temp": 24.2,
num": 36.0, "ts": "2026-06-15T13:17:36.893846+00:00"}
Publié → futurekawa/equateur/EQ01/mesures : {"entrepot": "EQ01", "temp": 24.2,
num": 36.0, "ts": "2026-06-15T13:17:38.921266+00:00"}
Publié → futurekawa/equateur/EQ01/mesures : {"entrepot": "EQ01", "temp": 24.2,
num": 36.0, "ts": "2026-06-15T13:17:40.948721+00:00"}
Publié → futurekawa/equateur/EQ01/mesures : {"entrepot": "EQ01", "temp": 24.2,
num": 36.0, "ts": "2026-06-15T13:17:42.976022+00:00"}
Publié → futurekawa/equateur/EQ01/mesures : {"entrepot": "EQ01", "temp": 24.2,
num": 36.0, "ts": "2026-06-15T13:17:45.003455+00:00"}
Publié → futurekawa/equateur/EQ01/mesures : {"entrepot": "EQ01", "temp": 24.2,
num": 36.0, "ts": "2026-06-15T13:17:47.026821+00:00"}
Publié → futurekawa/equateur/EQ01/mesures : {"entrepot": "EQ01", "temp": 24.2,
num": 36.0, "ts": "2026-06-15T13:17:49.054334+00:00"}
Publié → futurekawa/equateur/EQ01/mesures : {"entrepot": "EQ01", "temp": 24.2,
num": 38.0, "ts": "2026-06-15T13:17:51.081681+00:00"}
Publié → futurekawa/equateur/EQ01/mesures : {"entrepot": "EQ01", "temp": 24.3,
num": 40.0, "ts": "2026-06-15T13:17:53.109046+00:00"}
Publié → futurekawa/equateur/EQ01/mesures : {"entrepot": "EQ01", "temp": 24.4,
num": 43.0, "ts": "2026-06-15T13:17:55.136588+00:00"}
Publié → futurekawa/equateur/EQ01/mesures : {"entrepot": "EQ01", "temp": 24.5,
num": 46.0, "ts": "2026-06-15T13:17:57.163850+00:00"}
```

## 4. Plans de tests et résultats

### 4.1 Stratégie de tests

| Niveau              | Outil               | Périmètre                               |
|---------------------|---------------------|-----------------------------------------|
| Tests unitaires     | Jest (Node.js)      | Règles métier, seuils, FIFO, HTTP       |
| Tests d'intégration | Script shell + curl | Health checks API en conteneur          |
| Tests API           | Postman             | Endpoints REST (lots, mesures, alertes) |
| Tests end-to-end    | Manuel              | IoT → MQTT → BDD → API → Frontend       |

### 4.2 Tests unitaires — Backend pays (26 tests)

Commande : npm test dans backend-pays/

| Suite           | Tests                                  | Résultat |
|-----------------|----------------------------------------|----------|
| alertes.test.js | 9 tests — seuils IoT par pays          | ✅ PASS   |
| mesures.test.js | 7 tests — GET/POST mesures, hors plage | ✅ PASS   |
| fifo.test.js    | 3 tests — tri FIFO par date_stockage   | ✅ PASS   |
| lots.test.js    | 8 tests — CRUD lots, statuts, health   | ✅ PASS   |

Résultats : 27/27 tests passent en 3.5s

```
PASS tests/fifo.test.js
Logique FIFO – tri par date de stockage ASC
  ✓ ✓ Le lot le plus ancien apparaît en premier (37 ms)
  ✓ ✓ Liste vide retourne tableau vide (9 ms)
  ✓ ✓ Un seul lot retourne tableau d'un élément (5 ms)

PASS tests/lots.test.js
GET /health
  ✓ ✓ Répond 200 avec status ok (35 ms)
GET /lots
  ✓ ✓ Retourne liste FIFO (plus ancien en premier) (9 ms)
POST /lots
  ✓ ✓ Crée un lot et retourne 201 (21 ms)
  ✓ ✗ Retourne 404 si entrepôt inconnu (5 ms)
GET /lots – format réponse
  ✓ ✓ Retourne toujours un tableau (même vide) (8 ms)
GET /lots/:id
  ✓ ✓ Retourne un lot existant (5 ms)
  ✓ ✗ Retourne 404 si lot inexistant (4 ms)
PUT /lots/:id/statut
  ✓ ✓ Met à jour le statut en perime (6 ms)
  ✓ ✗ Retourne 404 si lot inexistant (6 ms)

Test Suites: 4 passed, 4 total
Tests:       27 passed, 27 total
Snapshots:   0 total
Time:        3.555 s
Ran all test suites.
```

## Cas de test clés — Seuils IoT

| Cas                         | Pays     | Entrée     | Attendu       | Résultat |
|-----------------------------|----------|------------|---------------|----------|
| Conditions idéales Brésil   | Brésil   | 29°C / 55% | ok: true      | ✓        |
| Limite haute acceptable     | Brésil   | 32°C / 57% | ok: true      | ✓        |
| Température trop haute      | Brésil   | 33°C / 55% | tempOk: false | ✓        |
| Humidité trop basse         | Brésil   | 29°C / 52% | humOk: false  | ✓        |
| Conditions idéales Équateur | Équateur | 31°C / 60% | ok: true      | ✓        |
| Température trop basse      | Équateur | 27°C / 60% | tempOk: false | ✓        |
| Conditions idéales Colombie | Colombie | 26°C / 80% | ok: true      | ✓        |
| Humidité trop haute         | Colombie | 26°C / 83% | humOk: false  | ✓        |
| Pays inconnu                | —        | n/a        | ok: false     | ✓        |

## Cas de test clés — Logique FIFO

| Cas                   | Données                    | Attendu                | Résultat |
|-----------------------|----------------------------|------------------------|----------|
| Tri lots par date ASC | 3 lots à dates différentes | Plus ancien en premier | ✓        |
| Liste vide            | 0 lots                     | Tableau vide           | ✓        |

| Cas        | Données | Attendu              | Résultat |
|------------|---------|----------------------|----------|
| Lot unique | 1 lot   | Tableau d'un élément | ✓        |

### 4.3 Tests unitaires — Backend siège (12 tests)

Commande : `npm test` dans `siège/api/`

| Suite                      | Tests                                     | Résultat |
|----------------------------|-------------------------------------------|----------|
| <code>siège.test.js</code> | 12 tests — agrégation, résilience, routes | ✓ PASS   |

Résultats : 12/12 tests passent en 2.8s

```
PASS tests/siège.test.js
  GET /health
    ✓ retourne status ok avec les pays configurés (41 ms)
  GET /siège/stocks
    ✓ agrège les stocks de tous les pays quand tous sont disponibles (11 ms)
    ✓ retourne une réponse partielle si un pays est down (timeout) (6 ms)
    ✓ retourne une réponse partielle si un pays est injoignable (ECONNREFUSED) (8 ms)
  GET /siège/stocks/:pays
    ✓ retourne les stocks d'un pays valide (6 ms)
    ✓ retourne 404 pour un pays inconnu (8 ms)
  GET /siège/mesures/:pays/:lot_id
    ✓ délègue vers l'API pays et retourne les mesures (7 ms)
    ✓ retourne 404 pour un pays inconnu (6 ms)
    ✓ retourne 503 si l'API pays est down (7 ms)
  GET /siège/alertes
    ✓ agrège les alertes de tous les pays (5 ms)
    ✓ retourne réponse partielle si un pays est en erreur (7 ms)
  Routes inconnues
    ✓ retourne 404 avec un message explicite (4 ms)

Test Suites: 1 passed, 1 total
Tests:      12 passed, 12 total
Snapshots:  0 total
Time:       1.642 s, estimated 2 s
Ran all test suites.
```

### Cas de test clés — Résilience siège

| Cas                       | Simulation                | Attendu                    | Résultat |
|---------------------------|---------------------------|----------------------------|----------|
| Tous les pays disponibles | axios mock → 200          | 3 pays OK                  | ✓        |
| Un pays en timeout        | axios mock → timeout      | Réponse partielle 2/3      | ✓        |
| Un pays ECONNREFUSED      | axios mock → ECONNREFUSED | Réponse partielle 2/3      | ✓        |
| Pays inconnu dans l'URL   | GET /siège/stocks/france  | 404                        | ✓        |
| Route inconnue            | GET /siège/inconnu        | 404 + message + lien /docs | ✓        |

### 4.4 Tests d'intégration — Health checks CI

Commande : `bash test-cicd/health-check.sh`

===== FutureKawa — Health Check CI =====

```
bresil-api (/health:3000) PASS {"status":"ok","pays":"bresil"}
equateur-api (/health:3000) PASS {"status":"ok","pays":"equateur"}
colombie-api (/health:3000) PASS {"status":"ok","pays":"colombie"}
siege-api (/health:8000) PASS {"status":"ok","service":"siege","version":"1.0.0","pays_
bresil","equateur","colombie"],"timeout_ms":3000}
```

=====

Resultats : 4 PASS | 0 FAIL

Tous les services sont operationnels.

=====

| Service      | Endpoint    | Résultat CI |
|--------------|-------------|-------------|
| bresil-api   | GET /health | ✓ PASS      |
| equateur-api | GET /health | ✓ PASS      |
| colombie-api | GET /health | ✓ PASS      |
| siege-api    | GET /health | ✓ PASS      |

## 4.5 Tests API — Collection Postman / Swagger

Fichier : backend-pays/postman/FutureKawa.postman\_collection.json

**FutureKawa — Seed Lots - Run results**

Run AgainNew RunAutomate RunShare

Ran today at 10:15:46 AM · View all runs

| Source | Environment | Iterations | Duration | All tests | Errors | Avg. Resp. Time |
|--------|-------------|------------|----------|-----------|--------|-----------------|
| Runner | none        | 1          | 1s 862ms | 0         | 0      | 34 ms           |

All TestsPassed (0)Failed (0)Skipped (0)Errors (0)Console LogView Summary

1

**POST** Équateur / POST LOT-EQ-2025-008 (EQ02 — conforme)

http://localhost:8002/lots

No tests found

201 • 8 ms • 459 B •

**POST** Équateur / POST LOT-EQ-2025-014 (EQ02 — en alerte)

http://localhost:8002/lots

No tests found

201 • 10 ms • 466 B •

**PUT** Équateur / PUT → périmé (LOT-EQ-2024-005)

http://localhost:8002/lots/LOT-EQ-2024-005/statut

No tests found

200 • 30 ms • 452 B •

**PUT** Équateur / PUT → en\_alerte (LOT-EQ-2025-014)

http://localhost:8002/lots/LOT-EQ-2025-014/statut

No tests found

200 • 7 ms • 462 B •

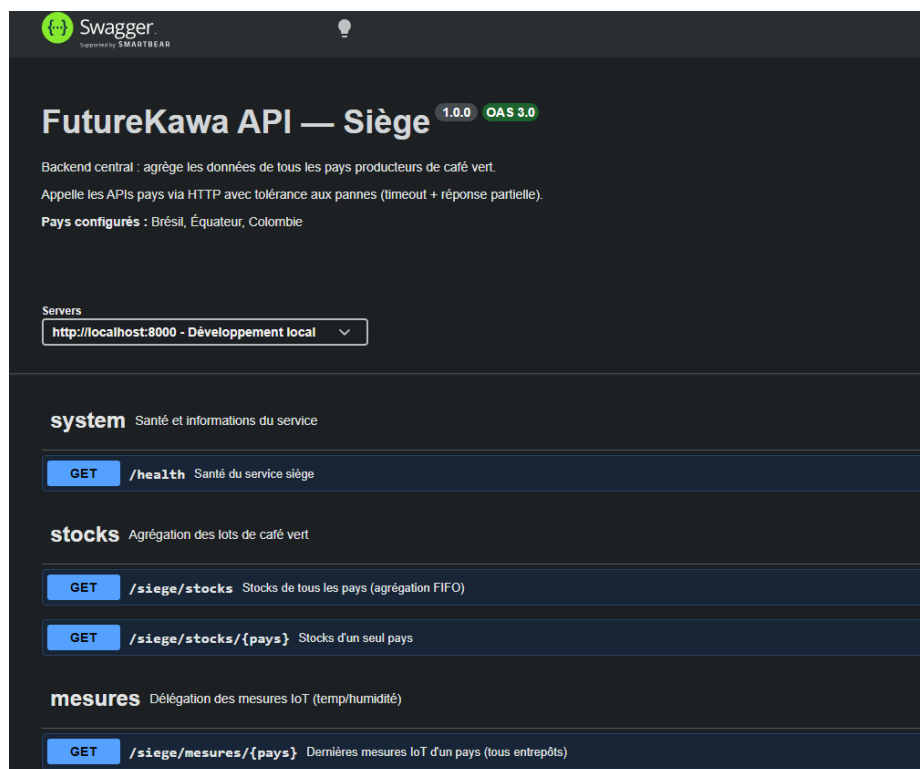
**GET** Équateur / GET /lots FIFO Équateur

http://localhost:8002/lots

No tests found

200 • 4 ms • 1.623 KB •

| Groupe        | Requêtes | Scénarios testés                                             |
|---------------|----------|--------------------------------------------------------------|
| Health        | 1        | GET /health → 200                                            |
| Lots Brésil   | 10       | POST (création), PUT (statut), GET (liste FIFO, détail)      |
| Lots Équateur | 8        | Idem Équateur                                                |
| Lots Colombie | 10       | Idem Colombie                                                |
| Mesures       | 3        | GET /mesures/:entrepot par pays                              |
| Siège         | 3        | GET /siege/stocks, /siege/stocks?pays=bresil, /siege/alertes |



## Cas de test FIFO

Procédure : Insérer 3 lots avec des dates différentes et vérifier l'ordre retourné.

```
// POST LOT-BR-2024-001 — date: 2024-06-01
// POST LOT-BR-2025-007 — date: 2026-01-05
// POST LOT-BR-2024-042 — date: 2025-09-01

// GET /lots — résultat attendu (tri ASC date_stockage)
[
  { "id": "LOT-BR-2024-001", "date_stockage": "2024-06-01..." },
  { "id": "LOT-BR-2024-042", "date_stockage": "2025-09-01..." },
  { "id": "LOT-BR-2025-007", "date_stockage": "2026-01-05..." }
]
```

## Cas de test alerte hors plage

Procédure : Le consumer MQTT reçoit une mesure hors seuil → vérifier la création d'une alerte et l'envoi email.

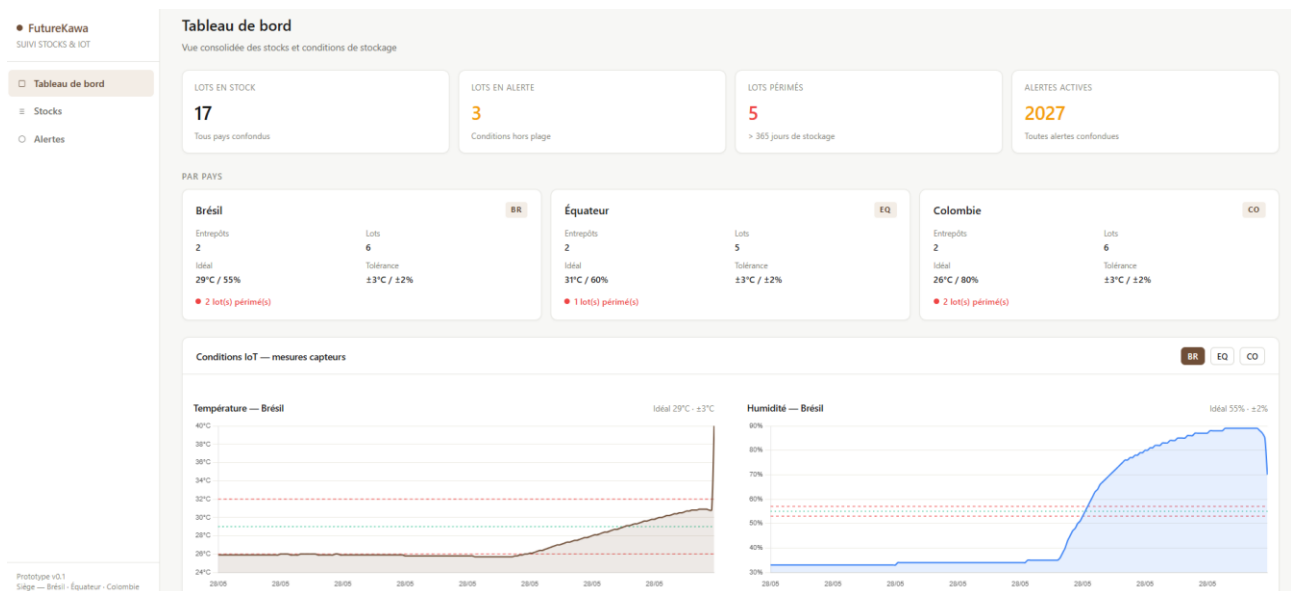
12. Publier sur futurekawa/bresil/BR01/mesures : {"temp": 35, "hum": 54, "entrepot": "BR01"}
13. GET /alertes → doit contenir une alerte type: hors\_plage
14. Mailhog UI (http://localhost:8025) → doit contenir l'email d'alerte



## 4.6 Tests end-to-end

Scénario complet validé :

15. Arduino branché, mqtt.py lancé
16. Mesures publiées sur futurekawa/bresil/BR01/mesures
17. Consumer insère en BDD (mesures table)
18. Alerte créée si hors plage
19. GET /siege/stocks retourne les lots
20. GET /siege/alertes retourne les alertes
21. Frontend affiche Dashboard + graphiques

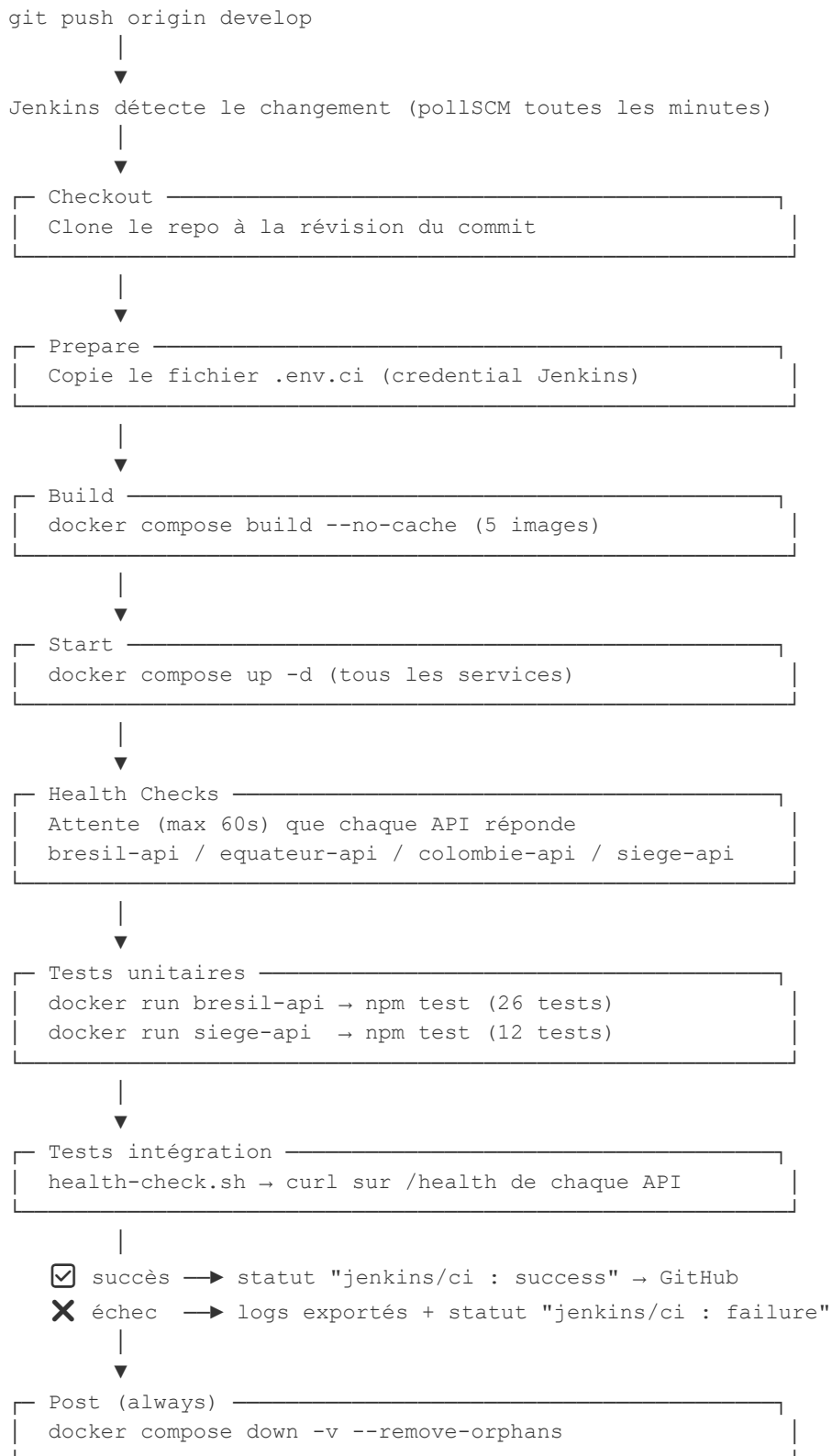


## 4.7 Anomalies rencontrées et corrections

| Anomalie                                                              | Cause                                                             | Correction                                                |
|-----------------------------------------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------|
| Route /alertes retournait les alertes des 3 pays pour chaque instance | Absence de filtre WHERE e.pays = ?                                | Ajout du filtre pays dans la requête                      |
| Bind mount mosquitto.conf échouait en CI                              | Docker socket : host ne trouve pas les fichiers du volume Jenkins | Config Mosquitto générée inline via entryptpoint          |
| Tests Jest : 3 tests échouaient                                       | Mocks pool.query insuffisants (1 mock pour 3 appels)              | Ajout des mocks manquants pour chaque appel query         |
| Frontend montrait données des 3 pays pour une alerte BR01             | Enrichissement incorrect du pays dans le siège                    | Filtre par r.pays lors de la transformation de la réponse |

## 5. Pipeline CI/CD Jenkins

### 5.1 Architecture du pipeline





## 5.2 Preuve d'exécution

Jenkins / FuturKawa

Status

Configure

Scan Pipeline Multibranches Now

Scan Pipeline Multibranches Log

Pipeline Multibranches Events

Delete Pipeline Multibranches

Historique des constructions

Relations entre les builds

Vérifier les empreintes numériques

Renommer

Pipeline Syntax

Identifiants

File d'attente des constructions

File d'attente des constructions vide

État du lanceur de compilations  
(0 of 2 executors busy)

FuturKawa

Branches (12)

| S | M | Name                 | Dernier succès | Dernier échec | Dernière durée |
|---|---|----------------------|----------------|---------------|----------------|
| ✓ | ☀ | datas                | 1 h 26 mn #8   | s. o.         | 7.5 s          |
| ✓ | ☀ | demo-test            | 1 h 40 mn #2   | s. o.         | 6.9 s          |
| ✓ | ☁ | develop              | 2 h 5 mn #9    | 2 h 24 mn #6  | 2 mn 50 s      |
| ✓ | ☀ | develop-correctifs   | 4 h 28 mn #1   | s. o.         | 14 s           |
| ✓ | ☁ | etape6-cicd          | 4 h 28 mn #4   | 20 j #2       | 18 s           |
| ✓ | ☀ | feature/mqtt-capteur | 4 h 28 mn #1   | s. o.         | 42 s           |
| ✓ | ☀ | feature-db           | 4 h 28 mn #1   | s. o.         | 25 s           |
| ✓ | ☀ | fix/ci-ports         | 4 h 28 mn #1   | s. o.         | 30 s           |
| ✓ | ☀ | Frontend             | 4 h 28 mn #1   | s. o.         | 31 s           |
| ✓ | ☁ | main                 | 2 h 7 mn #3    | 4 h 28 mn #1  | 2 mn 57 s      |
| ✓ | ☀ | mqtt                 | 4 h 28 mn #1   | s. o.         | 15 s           |

test: ajout test format reponse GET /lots #23

Merged Francoistlb merged 1 commit into develop from demo-test 2 hours ago

Conversation 0 Commits 1 Checks 0 Files changed 1

Francoistlb commented 2 hours ago

No description provided.

Mention @copilot in a comment to make changes to this pull request.

test: ajout test format reponse GET /lots ✓ 5f8ff24

Francoistlb merged commit 888e3f7 into develop 2 hours ago  
1 check passed

View details Revert

Au lancement d'une Pull Request, le build démarre automatique – Build #9



Jenkins / FuturKawa / develop

Status

Changes

Lancer un build

Voir la configuration

Pipeline Syntax

Log du dernier accès à Git

develop

Full project name: FuturKawa/develop

## Liens permanents

- [Dernier build \(#9\), il y a 2 h 5 mn](#)
- [Dernier build stable \(#9\), il y a 2 h 5 mn](#)
- [Dernier build avec succès \(#9\), il y a 2 h 5 mn](#)
- [Dernier build en échec \(#6\), il y a 2 h 24 mn](#)
- [Dernier build non réussi \(#6\), il y a 2 h 24 mn](#)
- [Dernier build complété \(#9\), il y a 2 h 5 mn](#)

Builds >

Filter

Today

- ✓ #9 12:23
- ✓ #8 12:20
- ✓ #7 12:08
- ✗ #6 12:04
- ✗ #5 11:57



Jenkins / FuturKawa / develop / #9

```
Cloning the remote Git repository
Cloning with configured refsspecs honoured and without tags
Cloning repository https://github.com/Francoistlb/mspr_tpre814_futurekawa.git
> git init /var/jenkins_home/workspace/FuturKawa_develop@2 # timeout=10
Fetching upstream changes from https://github.com/Francoistlb/mspr_tpre814_futurekawa.git
> git --version # timeout=10
> git --version # 'git version 2.47.3'
> git fetch --no-tags --force --progress -- https://github.com/Francoistlb/mspr_tpre814_futurekawa.git +refs/heads/*:refs
> git config remote.origin.url https://github.com/Francoistlb/mspr_tpre814_futurekawa.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
Checking out Revision 888e3f79a1dbedba31095de005623b376d63d8f3 (develop)
> git config core.sparsecheckout # timeout=10
> git checkout -f 888e3f79a1dbedba31095de005623b376d63d8f3 # timeout=10
Commit message: "Merge pull request #23 from Francoistlb/dermo-test"
> git rev-list --no-walk a004ff479b99f8b1cb66c115a766d80d817c1d07 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
```



Jenkins

/ FuturKawa

/ develop

/ #9

```
[Pipeline] sh
+ docker compose -f docker-compose.yml --env-file .env.ci images -q bresil-api
+ docker run --rm -e PAYS=bresil -e DB_PATH=/tmp/test.db -e MQTT_HOST=localhost -e MQTT_PORT=1883 -e MQ
RESPONSABLE_EMAIL=test@futurekawa.com fbe8d1c8231ec90a49160e1f15870fab836c4897933cc114b0981848bf198388

> backend-pays@1.0.0 test
> jest --verbose --ci --forceExit

PASS tests/alertes.test.js
  Seuils Brésil (29°C / 55%)
    ✓ ✓ Conditions idéales (15 ms)
    ✓ ✓ Limite haute acceptable (32°C / 57%) (2 ms)
    ✓ ✗ Température trop haute (33°C) (1 ms)
    ✓ ✗ Humidité trop basse (52%) (2 ms)
  Seuils Équateur (31°C / 60%)
    ✓ ✓ Conditions idéales (2 ms)
    ✓ ✗ Température trop basse (27°C) (2 ms)
  Seuils Colombie (26°C / 80%)
    ✓ ✓ Conditions idéales (2 ms)
    ✓ ✗ Humidité trop haute (83%) (2 ms)
  Pays inconnu
    ✓ ✗ Retourne ok: false (4 ms)

console.log
  ◇ injected env (0) from .env // tip: 🐞 custom filepath { path: '/custom/path/.env' }

    at _log (node_modules/dotenv/lib/main.js:131:11)

console.log
  ◇ injected env (0) from .env // tip: 🐞 enable debugging { debug: true }

    at _log (node_modules/dotenv/lib/main.js:131:11)
```



Jenkins

/ FuturKawa

/ develop

/ #9

```
GET /health
  ✓ ✓ Répond 200 avec status ok (43 ms)
GET /lots
  ✓ ✓ Retourne liste FIFO (plus ancien en premier) (4 ms)
POST /lots
  ✓ ✓ Crée un lot et retourne 201 (15 ms)
  ✓ ✗ Retourne 404 si entrepôt inconnu (4 ms)
GET /lots - format réponse
  ✓ ✓ Retourne toujours un tableau (même vide) (5 ms)
GET /lots/:id
  ✓ ✓ Retourne un lot existant (4 ms)
  ✓ ✗ Retourne 404 si lot inexistant (5 ms)
PUT /lots/:id/statut
  ✓ ✓ Met à jour le statut en perime (6 ms)
  ✓ ✗ Retourne 404 si lot inexistant (5 ms)

Test Suites: 4 passed, 4 total
Tests:       27 passed, 27 total
Snapshots:   0 total
Time:        3.4 s
Ran all test suites.

Force exiting Jest: Have you considered using `--detectOpenHandles` to detect async operations that kept running after all tests finished?

[Pipeline] sh
+ docker compose -f docker-compose.yml --env-file .env.ci images -q siege-api
+ docker run --rm -e BRESIL_API_URL=http://localhost:8001 -e EQUATEUR_API_URL=http://localhost:8002 -e COLOMBIE_API_URL=http://localhost:8003
7fd7164f62ff6f0399926a08e8b658f86adcc1b3df50477abd281e64ef13721 npm test -- --ci --forceExit

> futurekawa-siege-api@1.0.0 test
> jest --runInBand --forceExit --ci --forceExit
```



```
"html_url": "https://github.com/Francoistlb",
"followers_url": "https://api.github.com/users/Francoistlb/followers",
"following_url": "https://api.github.com/users/Francoistlb/following{/other_user}",
"gists_url": "https://api.github.com/users/Francoistlb/gists{/gist_id}",
"starred_url": "https://api.github.com/users/Francoistlb/starred{/owner}/{repo}",
"subscriptions_url": "https://api.github.com/users/Francoistlb/subscriptions",
"organizations_url": "https://api.github.com/users/Francoistlb/orgs",
"repos_url": "https://api.github.com/users/Francoistlb/repos",
"events_url": "https://api.github.com/users/Francoistlb/events{/privacy}",
"received_events_url": "https://api.github.com/users/Francoistlb/received_events",
"type": "User",
"user_view_type": "public",
"site_admin": false
}
}
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timeout
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

## 5.3 Particularités CI

### Docker socket pattern :

Jenkins tourne en conteneur et partage le socket Docker du host (/var/run/docker.sock). Les bind mounts de fichiers depuis le workspace Jenkins ne sont pas possibles car le daemon Docker du host ne peut pas résoudre les chemins internes au volume Jenkins. Solution : configuration Mosquitto générée inline via entryptpoint.

### Isolation des builds :

Le COMPOSE\_PROJECT\_NAME = "futurekawa-ci-\${BUILD\_NUMBER}" garantit que chaque build crée ses propres réseaux et conteneurs, sans conflit avec la stack locale ou les builds parallèles.

---

## 6. Sécurité — OWASP API Top 10

| Risque OWASP                                           | Mesure appliquée                                                                                                |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| API1 — Broken Object Level Authorization               | Chaque endpoint filtre par pays via variable d'environnement — impossible d'accéder aux données d'un autre pays |
| API2 — Broken Authentication                           | Pas d'authentification en v1 (démon locale). En production : JWT ou API key à ajouter                           |
| API3 — Broken Object Property Level                    | Les routes n'exposent que les champs nécessaires (pas de SELECT * non contrôlé)                                 |
| API4 — Unrestricted Resource Consumption               | Timeout 3s par pays côté siège. Limit paramètre sur /mesures?limit=200 (max 500)                                |
| API5 — Broken Function Level Authorization             | Pas de routes d'administration exposées                                                                         |
| API6 — Unrestricted Access to Sensitive Business Flows | Identifiant unique de lot — doublon rejeté par contrainte UNIQUE BDD                                            |
| API7 — Server Side Request Forgery                     | Le siège ne fait des requêtes qu'aux URLs configurées en .env — pas d'URL externe dynamique                     |
| API8 — Security Misconfiguration                       | CORS activé côté siège. Mosquitto en allow_anonymous uniquement en dev (à restreindre en production)            |
| API9 — Improper Inventory Management                   | Swagger UI disponible sur /docs pour chaque API — documentation exhaustive des endpoints                        |
| API10 — Unsafe Consumption of APIs                     | Validation Pydantic / express.json() sur tous les body. Requêtes SQL paramétrées (pas de concaténation)         |

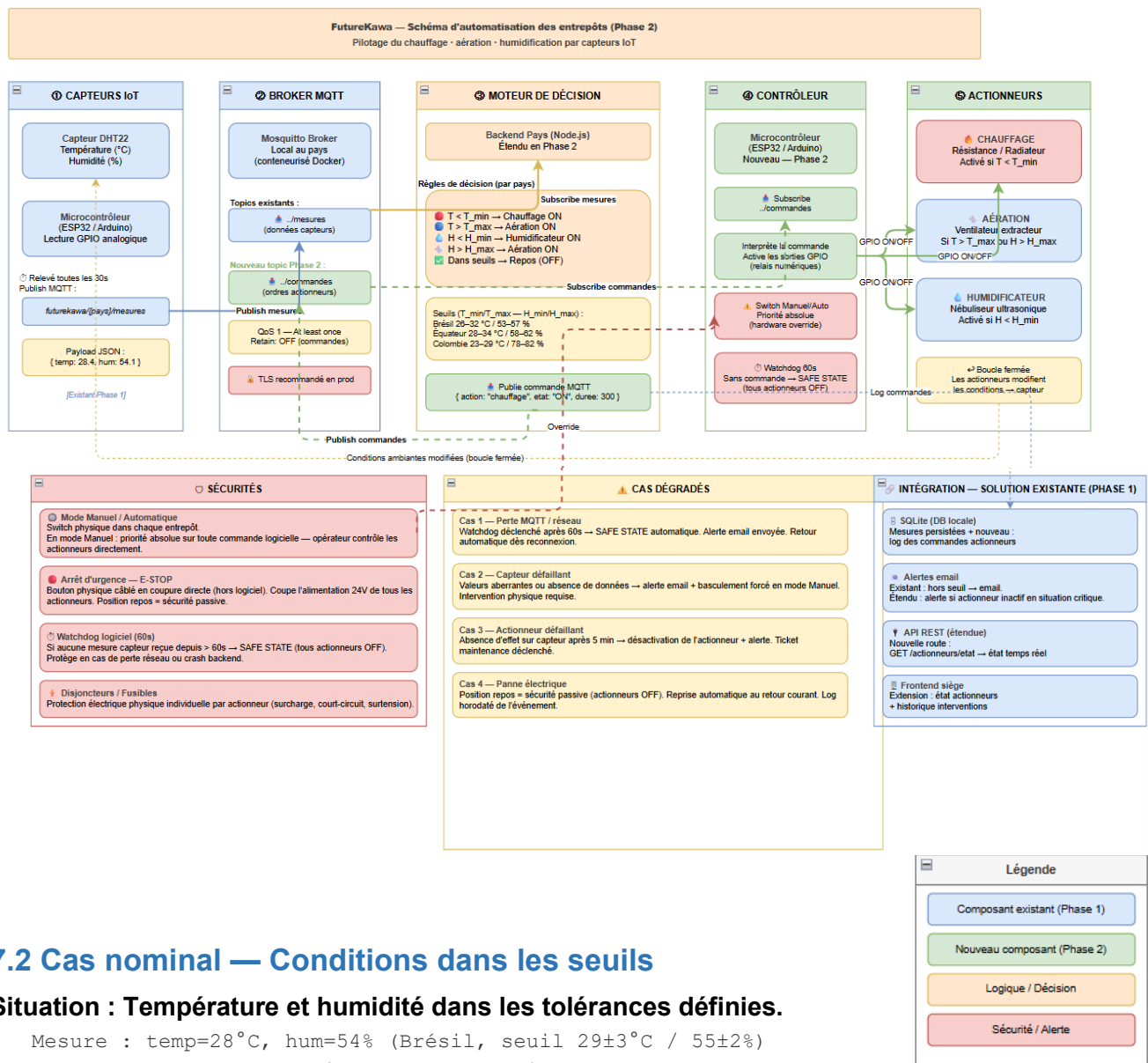
### Points de vigilance production :

- Activer l'authentification MQTT (username/password dans mosquitto.conf)
  - Ajouter authentification JWT sur les APIs REST
  - Restreindre CORS aux domaines autorisés
  - Passer de SQLite à PostgreSQL pour la concurrence multi-processus
-

## 7. Schéma d'automatisation — Phase 2

### 7.1 Vision générale

La phase 2 vise à automatiser le contrôle des conditions de stockage : les capteurs déclenchent des actionneurs (climatisation, humidificateur, ventilation) en fonction des mesures et des règles métier.



### 7.2 Cas nominal — Conditions dans les seuils

**Situation : Température et humidité dans les tolérances définies.**

Mesure : `temp=28°C, hum=54%` (Brésil, seuil  $29 \pm 3^\circ\text{C}$  /  $55 \pm 2\%$ )  
→ Dans les seuils → AUCUNE action

État actionneurs :

- Climatisation : STANDBY
- Humidificateur : STANDBY
- Ventilation : STANDBY

Actions automatiques : Aucune. Enregistrement de la mesure en BDD. Statut lot = conforme.

## 7.3 Cas dégradé — Température trop haute

### Situation : Température dépasse le seuil maximum.

Mesure : temp=34°C, hum=55% (Brésil, max autorisé 32°C)  
→ Hors seuil température

Actions automatiques (dans l'ordre) :

1. Alerte email → responsable entrepôt (immédiat)
2. Ventilation : ON (refroidissement passif)
3. Attente 5 min → nouvelle mesure
4. Si temp > 32°C : Climatisation : ON (puissance 50%)
5. Attente 10 min → nouvelle mesure
6. Si temp > 35°C : Climatisation : ON (puissance 100%) + alerte URGENCE

Statut lot : → en\_alerte

## 7.4 Cas dégradé — Panne capteur

### Situation : Aucune mesure reçue depuis X minutes.

Timeout capteur : aucune mesure depuis 10 minutes  
→ Panne capteur suspectée

Actions automatiques :

1. Alerte email : "Capteur BR01 hors ligne depuis 10 min"
2. Maintien dernier état des actionneurs connu
3. Passage en mode MANUEL forcé (sécurité)
4. Notification sur le dashboard (bannière rouge)

Résolution :

- Opérateur vérifie physiquement le capteur
- Redémarrage manuel ou remplacement
- Retour en mode AUTO via l'interface

## 7.5 Modes de fonctionnement

| Mode    | Description                                          | Déclenchement                    |
|---------|------------------------------------------------------|----------------------------------|
| AUTO    | Actionneurs pilotés par les règles métier            | Par défaut                       |
| MANUEL  | Opérateur contrôle chaque actionneur via l'interface | Bouton interface / panne capteur |
| URGENCE | Tous les actionneurs au maximum, alerte immédiate    | Dépassement seuil critique       |

## 7.6 Sécurités

| Sécurité                | Description                                                                |
|-------------------------|----------------------------------------------------------------------------|
| Timeout capteur         | Si aucune mesure depuis 10 min → mode MANUEL forcé                         |
| Seuil d'arrêt d'urgence | Température > 40°C → arrêt climatisation (risque condensation)             |
| Redondance capteurs     | En production, 2 capteurs par entrepôt (validation croisée)                |
| Journalisation          | Toutes les actions automatiques enregistrées avec timestamp et déclencheur |
| Mode manuel inviolable  | En mode MANUEL, aucune règle auto ne peut changer l'état des actionneurs   |

## 8. Questionnaire interview — Phase 2

*Ce questionnaire est destiné à cadrer le cahier des charges de la phase d'automatisation avec les responsables d'exploitation FutureKawa.*

### Objectifs et périmètre

22. Quels sont les 3 principaux problèmes que l'automatisation doit résoudre en priorité (qualité, pertes, coût énergétique) ?
23. Sur quels entrepôts souhaitez-vous déployer l'automatisation en premier ? Dans quel délai ?
24. Y a-t-il des entrepôts avec des contraintes spécifiques (bâtiments anciens, accès réseau limité, réglementation locale) ?

### Actionneurs et matériel

25. Quels actionneurs sont déjà présents dans les entrepôts (climatisation, ventilation, humidificateur) ? Sont-ils pilotables électroniquement ?
26. Disposez-vous d'un accès aux armoires électriques pour raccorder des relais de contrôle ?
27. Quel budget matériel est envisagé par entrepôt pour la phase 2 ?

### Seuils et règles métier

28. Les seuils actuels (ex.  $29^{\circ}\text{C} \pm 3^{\circ}\text{C}$  pour le Brésil) sont-ils validés par votre département qualité ? Peuvent-ils évoluer selon les variétés de café ou les saisons ?
29. Quelle tolérance de dépassement avant déclenchement d'un actionneur (immédiat ou après N minutes hors plage) ?
30. Comment gérez-vous aujourd'hui un pic de température en entrepôt (procédure manuelle) ?

### Sécurité et supervision

31. En cas de panne du système automatisé, qui est responsable de la reprise manuelle et sous quel délai ?
32. Souhaitez-vous un mode de supervision centralisé au siège permettant de prendre la main à distance sur les actionneurs d'un pays ?
33. Quelles sont les contraintes réglementaires locales sur le contrôle automatisé des installations électriques (Brésil, Équateur, Colombie) ?

### Déploiement et formation

34. Les équipes terrain ont-elles une formation technique suffisante pour maintenir des relais/automates ? Une formation est-elle envisageable ?
35. Quel niveau de connectivité réseau peut-on garantir dans les entrepôts (WiFi stable, GSM, Ethernet) ?
36. Quelle est la tolérance à l'indisponibilité du système automatisé (durée max acceptable sans supervision) ?